

설계과제 2

○ 개요

- 시스템 내 존재하는 동일(내용이 동일)한 파일을 찾고 삭제하는 프로그램

○ 팀 구성

- 개인별 프로젝트

○ 보고서 제출 방법 (강의계획서와 동일. #2 설계과제 명세에는 보고서 제출 방법을 삭제 예정. 강의계획서 참고)

- 설계과제는 “2_학번_버전번호.zip” (예. 2_20250000_V1.zip) 형태로 압축하여 classroom.google.com에 제출해야 함.
- 압축 파일 내 (1)구현보고서인 설계과제번호.hwp (예. 2.hwp, 2.hwp, 2.doc, 2.docx) (2)헤더파일 및 소스코드 등 해당 디렉토리에서 컴파일과 실행이 가능하도록 모든 파일(makefile, *.c, *.h 등 컴파일을 위한 모든 파일)을 포함시켜야 함. 단, 특정한 디렉토리에서 실행해야 할 경우는 디렉토리는 포함하지 않아 됨.
- 구현보고서인 “설계과제번호.hwp”에는 1. 과제개요(명세에서 주어진 개요를 그대로 쓰면 안됨. 자기가 분석하고, 구현한 내용 요약) 2. 구현플랫폼, 3. 상세설계(함수별 기능, 모든 함수의 프로토타입, 함수간 call graph (다양한 툴 이용 가능)), 4. 실행 결과, 5. 사용자 매뉴얼, 6. 소스코드 (절대 캡처 안됨, 텍스트 수준으로 복사하고 반드시 주석 포함)를 반드시 포함시켜야 함.

- | |
|---|
| <ol style="list-style-type: none">1. 과제 개요 // 위 개요를 더 상세하게 작성2. 구현 플랫폼 // glibc, gcc 버전 등3. 상세 설계 // 함수 기능별 흐름도(순서도, call graph), 함수 프로토타입 반드시 포함4. 실행 결과 // 자신의 컴퓨팅 환경의 디렉토리 구조 등을 고려하여 프로그램의 실행 결과 캡처 및 분석5. 사용자 매뉴얼 // 각 명령어와 옵션 기능과 사용법 설명 포함6. 소스코드 // 주석 |
|---|

- 제출한 압축 파일을 풀었을 때 해당 디렉토리에서 컴파일 및 실행이 되어야 함 (모든 프로그램은 해당 디렉토리에서 컴파일 및 실행이 되어야 함 (특정한 디렉토리에서 실행해야 할 경우는 제외). 해당 디렉토리에서 컴파일이나 실행되지 않을 경우, 설계과제 제출 방법(파일명, 컴파일 시에 포함되어야 할 파일에 포함되어야 할 파일 등)을 따르지 않는 경우 해당 과제 성적은 10점 감점.)
- 과제를 기한 내 새로 제출할 경우 기존 것은 삭제하지 않고 설계과제번호_학번_V2.zip 형태로 버전번호를 증가시키면 됨. 버전 번호는 대문자 V 와 함께 integer를 1부터 incremental 증가시키면서 부여하면 됨. (예. V1, V2, V3, ...). 첫 제출 시에도 버전 번호 V1(또는 V1.0)를 붙여야 하며, 두 번째는 V2(또는 V2.0)를 붙여 제출하면 됨. 단, 설계과제번호_학번_V1.zip 압축 파일 내 구현보고서에는 버전번호는 붙이지 말 것.

○ ssu_clean 프로그램 기본 사항

- ssu_clean은 지정한 디렉토리 내에서 사용자가 입력한 조건(파일 확장자, 파일 크기, 파일 생성 시간, 파일 수정 시간 등 파일 속성과 관련된 조건 등)에 맞춰 동일한(중복된) 정규 파일을 찾고 삭제하는 프로그램으로 동일한 정규 파일은 파일의 이름이 달라도 파일의 크기와 내용이 동일한 경우를 말함. ssu_clean은 텍스트 파일 뿐만 아니라 모든 형태의 바이너리 파일을 대상으로 함
- ssu_clean에 내장된 fmd5, fsha 명령어 (해시 함수는 md5나 sha 중 하나만 사용하고 구현)를 통해 주어진 인자의 시작 디렉토리 아래 모든 서브디렉토리까지 재귀적으로 탐색하여, 인자로 주어진 파일과 동일한(중복된) 파일을 찾아야 함
- 동일한(중복) 파일이 존재하는 경우 사용자가 입력한 옵션에 따라 동일한(중복) 파일 삭제할 수 있는 기능
- ssu_clean은 내장명령어(ssu_find-md5.c의 fmd5(또는 ssu_find-sha.c의 fsha), ssu_help.c의 help.c)를 위한 프로세스를 생성(fork())하고 이를 실행(exec() 류 함수)시켜 줌 (exit 명령어 제외)
- ssu_clean에 내장된 fmd5(또는 fsha 명령어)는 링크드 리스트로 동일한(중복) 파일 리스트로 그리고 BFS로 탐색 구현
- 참고로 리눅스 상에서 파일 경로의 최대 크기는 4,096 바이트이며, 파일 이름의 최대 크기는 255 바이트임
- system() 사용 불가

○ 설계 및 구현

- 가) ssu_clean
 - ✓ 프롬프트 출력
 - ✓ 프롬프트 모양 : “학번) ” 문자 출력 (예. 20201234))
 - ✓ 프롬프트에서는 ssu_clean의 내장 명령어인 fmd5(또는 fsha), exit, help 명령어만 실행
 - ✓ 이외 명령어 실행 시 자동으로 help를 실행시킨 것과 동일한 결과가 출력되어야 함
 - ✓ 엔터 키만 입력 시 프롬프트 재출력
- 나) 내장명령어 1, fmd5(또는 fsha)
 - ✓ 사용법 : fmd5 -s [FILE_EXTENSION] [MINSIZE] [MAXSIZE] [TARGET_DIRECTORY]
 - 지정한 디렉토리(TARGET_DIRECTORY) 아래 모든 디렉토리에서 지정한 파일 확장자(FILE_EXTENSION)을 갖는 특정 크기 사이(MINSIZE 이상 MAXSIZE 이하) 동일한(중복) 파일을 찾아 리스트로 출력
 - 지정한 디렉토리의 하위 디렉토리를 재귀적으로 탐색하여 동일한(중복) 파일을 찾음.
 - 단, 하위 디렉토리 탐색은 BFS 알고리즘을 사용

- “동일한(중복) 파일”은 파일의 해시값이 같은 정규 파일(텍스트, 바이너리 모두 포함)임
 - 파일별 해시값 비교 예시
 - 예시 1~4는 각각 ~/test/a, ~/test/b, ~/test/c, ~/test/d 파일의 내용을 보여줌
 - 예시 5는 ~/test/a, ~/test/b, ~/test/c, ~/test/d 파일의 크기를 보여줌
 - 예시 6은 ~/test/a, ~/test/b, ~/test/c, ~/test/d 파일의 해시값을 보여줌. a 파일과 d 파일의 해시값이 같다는 것을 알 수 있으며, 이는 동일한(중복) 파일임을 나타냄
 - 예제 1~6은 과제 구현을 위해 설명하는 예제이므로 해시값을 보여주는 것은 본 과제에서는 구현하지 않음)

(예시 1) a 파일 내용	
<pre> 1 Linux is a family of open-source Unix-like operating systems based on the Linux kernel, an operating system kernel first released on September 17, 1991, by Linus Torvalds. Linux is typically packaged in a Linux distribution. 2 Distributions include the Linux kernel and supporting system software and libraries, many of which are provided by the GNU Project. Many Linux distributions use the word "Linux" in their name, but the Free Software Foundation uses the name "GNU/Linux" to emphasize the importance of GNU software, causing some controversy. </pre>	
(예시 2) b 파일 내용 - a 파일과 비교하면 한 문자가 다른 파일	
<pre> 1 Pinux is a family of open-source Unix-like operating systems based on the Linux kernel, an operating system kernel first released on September 17, 1991, by Linus Torvalds. Linux is typically packaged in a Linux distribution. 2 Distributions include the Linux kernel and supporting system software and libraries, many of which are provided by the GNU Project. Many Linux distributions use the word "Linux" in their name, but the Free Software Foundation uses the name "GNU/Linux" to emphasize the importance of GNU software, causing some controversy. </pre>	
(예시 3) c 파일 내용 - a 파일과 비교하면 일부 내용이 추가된 파일	
<pre> 1 Linux is a family of open-source Unix-like operating systems based on the Linux kernel, an operating system kernel first released on September 17, 1991, by Linus Torvalds. Linux is typically packaged in a Linux distribution. 2 Distributions include the Linux kernel and supporting system software and libraries, many of which are provided by the GNU Project. Many Linux distributions use the word "Linux" in their name, but the Free Software Foundation uses the name "GNU/Linux" to emphasize the importance of GNU software, causing some controversy. 3 Popular Linux distributions include Debian, Fedora Linux, and Ubuntu. Commercial distributions include Red Hat Enterprise Linux and SUSE Linux Enterprise. Desktop Linux distributions include a windowing system such as X11 or Wayland, and a desktop environment such as GNOME or KDE Plasma. Distributions intended for servers may omit graphics altogether, or include a solution stack such as LAMP. Because Linux is freely redistributable, anyone may create a distribution for any purpose. </pre>	
(예시 4) d 파일 내용 - a 파일과 내용이 동일한 파일	
<pre> 1 Linux is a family of open-source Unix-like operating systems based on the Linux kernel, an operating system kernel first released on September 17, 1991, by Linus Torvalds. Linux is typically packaged in a Linux distribution. 2 Distributions include the Linux kernel and supporting system software and libraries, many of which are provided by the GNU Project. Many Linux distributions use the word "Linux" in their name, but the Free Software Foundation uses the name "GNU/Linux" to emphasize the importance of GNU software, causing some controversy. </pre>	
(예시 5) ~/test/a, b, c, d 파일의 크기 비교	(예시 6) ~/test/a, b, c, d 파일의 해시값 비교
<pre> su@su-virtual-machine:~/test-hash\$ ls -alH total 52 drwxrwxr-x 2 su su 4096 3월 22 16:15 . drwxr-xr-x 28 su su 4096 3월 22 16:15 .. -rw-rw-r-- 1 su su 547 3월 22 15:40 a -rw-rw-r-- 1 su su 547 3월 22 16:15 b -rw-rw-r-- 1 su su 1034 3월 22 15:33 c -rw-rw-r-- 1 su su 547 3월 22 16:15 d </pre>	<pre> su@su-virtual-machine:~/test-hash\$./ssu_md5 a 05f8d01138ae7136df8c4ddd3f01e378 su@su-virtual-machine:~/test-hash\$./ssu_md5 b 7c18e0cd5626833790abdb2cbb2fd1a4 su@su-virtual-machine:~/test-hash\$./ssu_md5 c aa50894bd6837536ad407d6297d1263a su@su-virtual-machine:~/test-hash\$./ssu_md5 d 05f8d01138ae7136df8c4ddd3f01e378 </pre>

- ✓ 따라서, 동일한(중복) 파일 탐색 시, 크기가 동일한 파일을 찾은 후 같은 크기를 가진 파일 중 각 파일의 해시값을 비교하여 동일한(중복) 파일 리스트를 만들
- ✓ 파일 이름이 다른 여러 개의 동일한(중복) 파일이 있을 수 있으며, 하나의 동일한(중복) 파일 리스트가 하나의 동일한(중복) 파일 세트가 됨. 예시 7에서 총 7개의 동일한(중복) 파일 리스트 세트를 확인할 수 있음 (Identical files #1 ~ #7, 총 7 개 세트)
- ✓ 여러 동일한(중복) 파일 세트가 존재할 때, 하나의 세트에는 여러 개의 동일한(중복) 파일들이 있을 수 있으며, 동일한(중복) 파일 세트는 ‘파일 크기’를 기준으로 작은 파일부터 큰 파일로 정렬해서 출력(파일 크기 오름차순). 파일의 크기가 같은 세트는 절대 경로를 기준으로 루트 디렉토리에서부터 경로가 (문자열 길이가) 짧은 세트부터 출력. 세트 내 최단 절대 경로의 길이가 동일하면 해시값의 사전순으로 세트를 정렬해서 출력. 예시 7 참고
 - MD5 해시값을 구하기 위해 MD5(openssl/md5.h)를 사용
 - ☞ Linux, Ubuntu : “sudo apt-get install libssl-dev”로 라이브러리 설치 필요
 - ☞ Linux, Fedora : “sudo dnf-get install libssl-devel”로 라이브러리 설치 필요
 - MD5 관련 함수를 사용하기 위해 컴파일 시 “-lcrypto” 옵션 필요

- MD5() 사용법은 OpenSSL 공식 문서(MD5 man page) 참고. 단, 직접 MD5_* 함수를 호출하는 방식보다 EVP 계열 인터페이스 사용
- 옵션 -s (a, m, c, i 옵션은 별도 설명)
- 탐색하고자 하는 파일의 크기 범위를 주는 옵션
- ✓ 두 번째 기본 인자 [FILE_EXTENSION]
 - 탐색할 파일의 확장자
 - "*" 입력 시, 모든 정규 파일을 대상으로 중복 파일 탐색
 - "*(확장자)" 입력 시, (확장자)인 정규 파일에서만 중복 파일 탐색 (예. *.jpg : jpg 확장자를 가진 파일에서만 중복 파일 탐색)
 - [FILE_EXTENSION] 인자에 입력이 없거나, "*", "*(확장자)" 외 다른 입력이 들어오면 에러 처리 후 프롬프트 출력
- ✓ 세 번째 기본 인자 [MINSIZE]
 - 탐색할 파일의 최소 크기
 - [MINSIZE] 인자에 바이트, KB, MB, GB 단위가 가능하며, 단위 미입력 시 기본 설정은 바이트 단위임 (단위는 소문자로 가능해야 하며, 공백으로 구분하지 않음)
 - [MINSIZE] 인자에 KB, MB, GB 단위는 실수도 가능해야 함 (실수의 소수점 이하 자리는 KB는 3자리 이상, MB는 6자리 이상, GB는 9자리 이상이 표시되면 됨. 그 이상의 자리는 의미 없음)
 - [MINSIZE] 인자에 '~'를 입력할 경우, [MAXSIZE] 이하인 모든 파일에서 중복 파일 탐색
 - [MINSIZE] 인자에 입력이 없거나, 숫자(실수 포함)나 '~' 외 다른 입력이 들어오면 에러 처리 후 프롬프트 출력
- ✓ 네 번째 기본 인자 [MAXSIZE]
 - 탐색할 파일의 최대 크기
 - [MAXSIZE] 인자에 바이트, KB, MB, GB 단위가 가능하며, 단위 미입력 시 기본 설정은 바이트 단위임 (단위는 소문자로 가능해야 하며, 공백으로 구분하지 않음)
 - [MAXSIZE] 인자에 KB, MB, GB 단위는 실수도 가능해야 함 (실수의 소수점 이하 자리는 KB는 3자리 이상, MB는 6자리 이상, GB는 9자리 이상이 표시되면 됨. 그 이상의 자리는 의미 없음)
 - [MAXSIZE] 인자에 '~'를 입력할 경우, [MINSIZE] 이상인 모든 파일에서 중복 파일 탐색
 - [MAXSIZE] 인자에 입력이 없거나, 숫자(실수 포함)나 '~' 외 다른 입력이 들어오면 에러 처리 후 프롬프트 출력
 - [MINSIZE]와 [MAXSIZE] 모두 '~'이면 크기 제한 없이 중복 파일 탐색
- ✓ 다섯 번째 기본 인자 [TARGET_DIRECTORY]
 - 탐색할 디렉토리 경로(절대경로와 ~(홈 디렉토리)를 포함한 상대경로 모두 가능해야 함)
 - 인자에 루트("/")를 입력할 경우, 파일 탐색 시 권한 문제가 발생할 수 있으므로 root 권한으로 실행해야 함
 - 루트 디렉토리부터 탐색 시, "proc"과 "run", "sys" 디렉토리는 반드시 제외하여 탐색을 진행
 - [TARGET_DIRECTORY] 인자에 입력이 없거나, 디렉토리가 아니거나, 존재하지 않는 디렉토리인 경우, 에러 처리 후 프롬프트 출력
- ✓ ssu_clean 명령어 실행 시 출력은 다음과 같음
 - 지정한 디렉토리 내에 중복 파일이 있는 경우, 각 중복 파일 리스트를 구분하여 한 세트씩 중복 파일 리스트 출력
 - 첫째 줄에는 각 중복 파일 리스트의 세트 번호와 파일 크기(바이트 단위), 해시값을 출력. 예시 7 참고
 - ☞ 파일 크기는 천 단위마다 ','로 구분해야 함
 - 다음 줄부터, 중복 파일 리스트 내의 인덱스 번호와 파일의 절대 경로, 마지막 수정 시간, 마지막 접근 시간을 출력함. 예시 7 참고
 - ☞ 세트 내 리스트 파일들의 출력순서는 파일 절대 경로가 짧은 것부터, 동일한 절대 경로는 아스키 코드 순서
 - ☞ 마지막 수정 시간과 접근 시간은 stat 구조체 이용
 - 위 과정을 반복하여 전체 중복 파일 리스트 출력
 - 지정한 디렉토리 내에 중복 파일이 없는 경우, "No duplicates in (TARGET_DIRECTORY의 절대경로)" 출력 후 프롬프트 출력. 예시 10 참고
 - 중복 파일 탐색 및 출력을 마치면 마지막 줄에 탐색 소요 시간 출력. 예시 7 참고
 - 중복 파일 리스트가 존재하는 경우 ">>"를 출력하고 사용자 입력 대기

(예시 7). 파일 탐색 결과 출력 - 지정한 디렉토리 내 모든 중복 파일 리스트 출력

```
su@su-virtual-machine:~/2022-1_LSP/P2$ ./ssu_clean
20220000> fmd5 -s * ~ ~ ~/test
---- Identical files #1 (5 bytes - 2b00042f7481c7b056c4b410d28f33cf) ----
[1] /home/su/test/aa/b/a.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/b/b/ccc.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 17:11:30)
[3] /home/su/test/aa/a/aaa/a.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 16:54:11)
[4] /home/su/test/aa/a/bbb/a.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 16:54:11)

---- Identical files #2 (10 bytes - a61b545ae32677047a21014d7af49fee) ----
[1] /home/su/test/1 (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:46:54)
[2] /home/su/test/2 (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:47:09)
[3] /home/su/test/a b (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:38:03)
```

```

---- Identical files #3 (16 bytes - 7109c5edcae94719a22fb9f4023b32b0) ----
[1] /home/su/test/out3.bin (mtime : 2022-01-10 10:53:28) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/out4.bin (mtime : 2022-01-10 10:53:28) (atime : 2022-03-23 16:54:11)
[3] /home/su/test/b/out3.bin (mtime : 2022-01-10 10:53:28) (atime : 2022-03-23 16:54:11)
[4] /home/su/test/b/out4.bin (mtime : 2022-01-10 10:53:28) (atime : 2022-03-23 16:54:11)

---- Identical files #4 (32 bytes - 79e8cbdab7a957d55f52fa6da06241be) ----
[1] /home/su/test/a.bin (mtime : 2022-01-19 12:42:25) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/b.bin (mtime : 2022-01-19 12:42:25) (atime : 2022-03-23 16:54:11)
[3] /home/su/test/c.bin (mtime : 2022-01-19 12:42:25) (atime : 2022-03-23 16:54:11)

---- Identical files #5 (32 bytes - b82aa750b2124c0b00668a267a5dc3bf) ----
[1] /home/su/test/a/different_a.bin (mtime : 2022-01-19 22:30:12) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/aa/different_b.bin (mtime : 2022-01-19 22:30:12) (atime : 2022-03-23 16:54:11)
[3] /home/su/test/b/different_c.bin (mtime : 2022-01-19 22:30:12) (atime : 2022-03-23 16:54:11)

---- Identical files #6 (1,219,664 bytes - 6596d99856f98ba9476b455bde8f3ca5) ----
[1] /home/su/test/sample1.o (mtime : 2022-02-08 17:36:52) (atime : 2022-03-22 17:31:45)
[2] /home/su/test/sample2.o (mtime : 2022-02-08 17:35:40) (atime : 2022-03-22 17:31:45)
[3] /home/su/test/test1 (copy).o (mtime : 2022-02-08 17:36:52) (atime : 2022-03-23 15:38:19)
[4] /home/su/test/test1.o (mtime : 2022-02-08 17:36:52) (atime : 2022-03-22 17:31:45)

---- Identical files #7 (1,885,256 bytes - 40b1c990a700fd241db3f09a402ff216) ----
[1] /home/su/test/example1.o (mtime : 2022-02-08 17:36:54) (atime : 2022-03-22 17:31:45)
[2] /home/su/test/example2.o (mtime : 2022-02-08 17:35:05) (atime : 2022-03-22 17:31:45)
[3] /home/su/test/example3.o (mtime : 2022-02-08 17:35:43) (atime : 2022-03-22 17:31:45)

Searching time: 0:018930(sec:usec)

>>

```

(예시 8). 파일 탐색 결과 출력 - 지정한 디렉토리 내 png 확장자 파일 중 중복 파일 리스트 출력

```

su@su-virtual-machine:~/2022-1_LSP/P2$ ./ssu_clean
20220000> fmd5 -s *.png ~ ~ /test
---- Identical files #1 (5 bytes - 2b00042f7481c7b056c4b410d28f33cf) ----
[1] /home/su/test/aa/b/a.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/b/b/c/c.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 17:11:30)
[3] /home/su/test/aa/a/aaa/a.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 16:54:11)
[4] /home/su/test/aa/a/bbb/a.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 16:54:11)

Searching time: 0:000259(sec:usec)

>>

```

(예시 9). 파일 탐색 결과 출력 - 지정한 디렉토리 내 32바이트 이상 1.8MB 이하 중복 파일 리스트 출력

```
20220000> fmd5 -s * 32 1.8MB ~/test
---- Identical files #1 (32 bytes - 79e8cbdab7a957d55f52fa6da06241be) ----
[1] /home/su/test/a.bin (mtime : 2022-01-19 12:42:25) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/b.bin (mtime : 2022-01-19 12:42:25) (atime : 2022-03-23 16:54:11)
[3] /home/su/test/c.bin (mtime : 2022-01-19 12:42:25) (atime : 2022-03-23 16:54:11)

---- Identical files #2 (32 bytes - b82aa750b2124c0b00668a267a5dc3bf) ----
[1] /home/su/test/a/different_a.bin (mtime : 2022-01-19 22:30:12) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/aa/different_b.bin (mtime : 2022-01-19 22:30:12) (atime : 2022-03-23 16:54:11)
[3] /home/su/test/b/different_c.bin (mtime : 2022-01-19 22:30:12) (atime : 2022-03-23 16:54:11)

---- Identical files #3 (1,219,664 bytes - 6596d99856f98ba9476b455bde8f3ca5) ----
[1] /home/su/test/sample1.o (mtime : 2022-02-08 17:36:52) (atime : 2022-03-22 17:31:45)
[2] /home/su/test/sample2.o (mtime : 2022-02-08 17:35:40) (atime : 2022-03-22 17:31:45)
[3] /home/su/test/test1 (copy).o (mtime : 2022-02-08 17:36:52) (atime : 2022-03-23 15:38:19)
[4] /home/su/test/test1.o (mtime : 2022-02-08 17:36:52) (atime : 2022-03-22 17:31:45)

Searching time: 0:010421(sec:usec)

>>
```

(예시 10). 파일 탐색 결과 출력 - 지정한 디렉토리 내 2MB 이상 파일 중 중복 파일이 없는 경우

```
20220000> fmd5 -s * 2mb ~ ~/test
No duplicates in /home/su/test
Searching time: 0:000148(sec:usec)

20220000>
```

✓ >> [SET_INDEX] [OPTION ...]

- 사용자가 선택한 [SET_INDEX]에 해당하는 중복 파일 리스트에서 [OPTION]에 따라 중복 파일 삭제
- 파일 삭제 후, 남아있는 중복 파일 리스트를 보여주고 ">>"를 출력한 후 사용자 입력 대기
- 남아있는 중복 파일 리스트가 없으면 프롬프트 출력
- "exit"를 입력하면 ">> Back to Prompt"를 출력하고 프롬프트 출력

(예시 11). >> exit 수행 결과

```
20220000> fmd5 -s * ~ 5 ~/test
---- Identical files #1 (5 bytes - 2b00042f7481c7b056c4b410d28f33cf) ----
[1] /home/su/test/aa/b/a.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/b/b/c.ccc.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 17:11:30)
[3] /home/su/test/aa/a/aaa/a.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 16:54:11)
[4] /home/su/test/aa/a/bbb/a.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 16:54:11)

Searching time: 0:000218(sec:usec)

>> exit
>> Back to Prompt
20220000>
```

- [SET_INDEX]

- 전체 중복 파일 리스트에서 삭제하고자 하는 중복 파일 리스트의 세트 번호
- [SET_INDEX] 인자에 입력이 없거나 범위를 벗어난 경우, 에러 처리 후 다시 입력 대기

- [OPTION]

- 선택한 세트에서 파일을 삭제하기 위한 옵션
- d, i, f, t 옵션을 구현해야 하며, 이외의 옵션이 들어오면 에러 처리 후 ">>"를 출력하고 사용자 입력 대기
- 옵션은 중복으로 사용하지 않음
- d [LIST_IDX] : 선택한 세트에서 [LIST_IDX]에 해당하는 파일 삭제
 - ☞ [LIST_IDX] 인자에 입력이 없거나 범위를 벗어난 경우, 에러 처리 후 ">>"를 출력하고 사용자 입력 대기
 - ☞ 삭제 후 예시 12와 와 같이 삭제한 파일의 절대 경로 출력

(예시 12). d 옵션 출력

```
20220000> fmd5 -s * ~ 9 ~/test
---- Identical files #1 (5 bytes - 2b00042f7481c7b056c4b410d28f33cf) ----
[1] /home/su/test/aa/b/a.png (mtime : 2022-03-15 01:20:08) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/b/b/ccc.png (mtime : 2022-03-15 01:20:08) (atime : 2022-03-23 17:11:30)
[3] /home/su/test/aa/a/aaa/a.png (mtime : 2022-03-15 01:20:08) (atime : 2022-03-23 16:54:11)
[4] /home/su/test/aa/a/bbb/a.png (mtime : 2022-03-15 01:20:08) (atime : 2022-03-23 16:54:11)

Searching time: 0:000213(sec:usec)

>> 1 d 3
"/home/su/test/aa/a/aaa/a.png" has been deleted in #1

---- Identical files #1 (5 bytes - 2b00042f7481c7b056c4b410d28f33cf) ----
[1] /home/su/test/aa/b/a.png (mtime : 2022-03-15 01:20:08) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/b/b/ccc.png (mtime : 2022-03-15 01:20:08) (atime : 2022-03-23 17:11:30)
[3] /home/su/test/aa/a/bbb/a.png (mtime : 2022-03-15 01:20:08) (atime : 2022-03-23 16:54:11)

>>
```

- i : 선택한 세트의 중복 파일 리스트에 있는 파일의 절대 경로를 하나씩 보여주면서 삭제 여부 확인 후 파일 삭제 또는 유지
 - ☞ 중복 파일 리스트 원소의 절대경로를 하나씩 보여주면서 삭제 여부 확인 메시지 출력
 - ☞ “y”나 “Y” 입력 시, 해당 파일 삭제, “n”나 “N” 입력 시, 해당 파일 유지
 - 이외의 입력이 들어오면 예외 처리 후 “>>” 출력하고 사용자 입력 대기
 - ☞ 모든 파일에 대해 삭제 여부 확인 완료하면 전체 중복 파일 리스트 출력
 - 선택한 세트의 중복 파일 리스트에 있는 파일을 모두 삭제하거나 하나만 남는 경우 중복 파일 리스트에서 제거됨

(예시 13). i 옵션 출력

```
20220000> fmd5 -s * 10 15 ~/test
---- Identical files #1 (10 bytes - a61b545ae32677047a21014d7af49fee) ----
[1] /home/su/test/1 (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:46:54)
[2] /home/su/test/2 (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:47:09)
[3] /home/su/test/a b (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:38:03)
[4] /home/su/test/a b (another copy) (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:38:19)
[5] /home/su/test/a b (copy) (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:38:19)

Searching time: 0:000226(sec:usec)

>> 1 i
Delete "/home/su/test/1"? [y/n] y
Delete "/home/su/test/2"? [y/n] N
Delete "/home/su/test/a b"? [y/n] Y
Delete "/home/su/test/a b (another copy)"? [y/n] n
Delete "/home/su/test/a b (copy)"? [y/n] y

---- Identical files #1 (10 bytes - a61b545ae32677047a21014d7af49fee) ----
[1] /home/su/test/2 (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:47:09)
[2] /home/su/test/a b (another copy) (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:38:19)

>> 1 i
Delete "/home/su/test/2"? [y/n] y
Delete "/home/su/test/a b (another copy)"? [y/n] y

20220000>
```

- f : 가장 최근에 수정한 파일을 남겨두고 나머지 중복 파일을 삭제
 - ☞ 삭제 후 예시 14와 같이 가장 최근에 수정한 파일의 절대 경로와 수정 시간 출력

☞ f 옵션 사용 시, 파일이 하나만 남기 때문에 중복 파일 리스트에서는 제거됨

(예시 14). f 옵션 출력

```
20220000> fmd5 -s * 32 1kb ~/test
---- Identical files #1 (32 bytes - 79e8cbdab7a957d55f52fa6da06241be) ----
[1] /home/su/test/a.bin (mtime : 2022-01-19 12:42:25) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/b.bin (mtime : 2022-01-19 12:42:25) (atime : 2022-03-23 16:54:11)
[3] /home/su/test/c.bin (mtime : 2022-01-19 12:42:25) (atime : 2022-03-23 16:54:11)

---- Identical files #2 (32 bytes - b82aa750b2124c0b00668a267a5dc3bf) ----
[1] /home/su/test/a/different_a.bin (mtime : 2022-01-19 22:30:12) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/aa/different_b.bin (mtime : 2022-01-19 22:30:12) (atime : 2022-03-23 16:54:11)
[3] /home/su/test/b/different_c.bin (mtime : 2022-01-19 22:30:12) (atime : 2022-03-23 16:54:11)

Searching time: 0:000373(sec:usec)

>> 1 f
Left file in #1 : /home/su/test/a.bin (2022-01-19 12:42:25)

---- Identical files #1 (32 bytes - b82aa750b2124c0b00668a267a5dc3bf) ----
[1] /home/su/test/a/different_a.bin (mtime : 2022-01-19 22:30:12) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/aa/different_b.bin (mtime : 2022-01-19 22:30:12) (atime : 2022-03-23 16:54:11)
[3] /home/su/test/b/different_c.bin (mtime : 2022-01-19 22:30:12) (atime : 2022-03-23 16:54:11)

>>
```

- t : 가장 최근에 수정한 파일을 남겨두고 나머지 중복 파일을 휴지통으로 이동. 휴지통은 간단하게 특정 디렉토리를 지정해서 구현. 즉, 휴지통으로 이동하는 것은 미리 지정한 디렉토리로 이동시키는 것임.

☞ 삭제 후 예시 15와 같이 가장 최근에 수정한 파일의 절대 경로와 수정 시간 출력

☞ t 옵션 사용 시, 파일이 하나만 남기 때문에 중복 파일 리스트에서는 제거됨

(예시 15). t 옵션 출력

```
20220000> fmd5 -s * ~ 10 ~/test
---- Identical files #1 (5 bytes - 2b00042f7481c7b056c4b410d28f33cf) ----
[1] /home/su/test/aa/b/a.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 16:54:11)
[2] /home/su/test/b/b/c.ccc.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 17:11:30)
[3] /home/su/test/aa/a/aaa/a.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 16:54:11)
[4] /home/su/test/aa/a/bbb/a.png (mtime : 2022-01-15 01:20:08) (atime : 2022-03-23 16:54:11)

---- Identical files #2 (10 bytes - a61b545ae32677047a21014d7af49fee) ----
[1] /home/su/test/1 (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:46:54)
[2] /home/su/test/2 (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:47:09)
[3] /home/su/test/a b (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:38:03)
[4] /home/su/test/a b (another copy) (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:38:19)
[5] /home/su/test/a b (copy) (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:38:19)

Searching time: 0:000249(sec:usec)

>> 1 t
All files in #1 have moved to Trash except "/home/su/test/aa/b/a.png" (2022-01-15 01:20:08)

---- Identical files #1 (10 bytes - a61b545ae32677047a21014d7af49fee) ----
[1] /home/su/test/1 (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:46:54)
[2] /home/su/test/2 (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:47:09)
[3] /home/su/test/a b (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:38:03)
[4] /home/su/test/a b (another copy) (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:38:19)
[5] /home/su/test/a b (copy) (mtime : 2022-01-23 15:37:54) (atime : 2022-03-23 15:38:19)
```

- 나) (md5 대신 sha로 구현하는 경우) 내장명령어 2, fsha
 - ✓ 사용법 : fsha -s [FILE_EXTENSION] [MINSIZE] [MAXSIZE] [TARGET_DIRECTORY]
 - SHA1 해시값을 구하기 위해 <openssl/sha.h>에 정의된 SHA1 관련 함수 사용
 - ☞ Linux, Ubuntu : “sudo apt-get install libssl-dev”로 라이브러리 설치 필요
 - ☞ Linux, Fedora : “sudo dnf-get install libssl-devel”로 라이브러리 설치 필요
 - SHA1 관련 함수를 사용하기 위해 컴파일 시 “-lcrypto” 옵션 필요
 - OpenSSL 공식 매뉴얼(SHA1 man page) 참고. 단, OpenSSL에서는 SHA1() 직접 사용보다 EVP_Digest 계열 인터페이스 사용.
 - ✓ 내장명령어 1, fmd5의 모든 실행결과와 동일한 형태로 출력(본 명세에서는 예시는 따로 보여주지 않음)
- 다) 내장명령어를 위한 멀티 옵션
 - ✓ 내장명령어 첫 번째 인자 -s 외 옵션으로 a, m, c, h 옵션을 구현해야 하며, -a, -m, -c 옵션은 반드시 -s 옵션 없이도 단독으로 사용해야 하며, h 옵션은 반드시 -s, -a, -m, -c와 함께 사용(-h 옵션 단독 사용 불가). 또한 a, m, c는 중복 사용 불가능
 - fmd5 -s [FILE_EXTENSION] [MINSIZE] [MAXSIZE] [TARGET_DIRECTORY]
 - fmd5 -sh [FILE_EXTENSION] [MINSIZE] [MAXSIZE] [TARGET_DIRECTORY]
 - fmd5 -s [FILE_EXTENSION] [MINSIZE] [MAXSIZE] [TARGET_DIRECTORY] -a [ATIME_FROM] [ATIME_TO]
 - fmd5 -s [FILE_EXTENSION] [MINSIZE] [MAXSIZE] [TARGET_DIRECTORY] -m [MTIME_FROM] [MTIME_TO]
 - fmd5 -s [FILE_EXTENSION] [MINSIZE] [MAXSIZE] [TARGET_DIRECTORY] -c [CTIME_FROM] [CTIME_TO]
 - fmd5 -sh [FILE_EXTENSION] [MINSIZE] [MAXSIZE] [TARGET_DIRECTORY] -a [ATIME_FROM] [ATIME_TO]
 - fmd5 -sh [FILE_EXTENSION] [MINSIZE] [MAXSIZE] [TARGET_DIRECTORY] -m [MTIME_FROM] [MTIME_TO]
 - fmd5 -sh [FILE_EXTENSION] [MINSIZE] [MAXSIZE] [TARGET_DIRECTORY] -c [CTIME_FROM] [CTIME_TO]
 - fmd5 -a [ATIME_FROM] [ATIME_TO] [TARGET_DIRECTORY]
 - fmd5 -m [MTIME_FROM] [MTIME_TO] [TARGET_DIRECTORY]
 - fmd5 -c [CTIME_FROM] [CTIME_TO] [TARGET_DIRECTORY]
 - ✓ - a (일정 액세스 기간) fmd5 -a [FROM] [TO] [TARGET_DIRECTORY]
 - 탐색하고자 하는 파일의 접근 시간을 범위((from to) [년:월:일:시] ~ [년:월:일:시])로 받아 이 시간 내(주어진 시간 포함) 파일을 탐색. 단, 앞의 인자 [년:월:일:시]가 뒤의 인자 [년:월:일:시]보다 더 오래된 시간이어야 함. 이 부분도 에러 처리 필수.
 - 년도(4자리로 처리)만 따로 줄 수도 있고, 월(01과 1 모두 인식 가능하게)까지만 줄 수도 있고, 일까지만 줄 수도 있고, 시까지만 인자로 줄 수 있게 구현
 - 앞의 인자와 [년:월:일:시]와 뒤의 인자 [년:월:일:시] 값을 모두 ~으로 주면 전체 시간을, 앞의 인자에 ~을 주면, 뒤 인자 시간까지, 뒤의 인자를 ~으로 주면, 앞의 시간부터 프로그램을 수행하는 현재까지를 의미. 예를 들어 2024:03:0:0의 경우 앞의 인자일 경우와 뒤의 인자일 경우 해석이 달라야 함.
 - ☞ 2024:0:0:0와 ~으로 인자를 줄 경우, 2024-01-01 ~ 현재 시간(프로그램 실행 시간) 의미
 - ☞ 2024:03:0:0와 ~으로 인자로 줄 경우, 2024-03-01 ~ 현재 시간(프로그램 실행 시간) 의미
 - ☞ ~과 2024:12:0:0을 인자로 줄 경우, 2024-12-31 이전 모든 파일
 - ☞ ~과 2024:4:0:0을 인자로 줄 경우, 2024-4-30 이전 모든 파일
 - ☞ 2024:0:0:0과 2024:4:0:0을 인자로 줄 경우, 2024-01-01 ~ 2024-4-30 이전 모든 파일
 - ☞ 이와 유사한 경우의 예는 학생들이 일관성 있게 룰을 만들어 구현
 - 아래 모든 시간 관련 옵션은 위와 동일하게 구현
 - 잘못된 년도, 잘못된 월, 잘못된 일, 잘못된 시간 표현은 모두 에러 처리를 해야 함.
 - ✓ - m (일정 수정 기간) fmd5 -m [FROM] [TO] [TARGET_DIRECTORY]
 - 탐색하고자 하는 파일의 수정 시간 범위(from~to)를 주는 옵션.
 - 년도만 따로 줄 수도 있고, 월까지만 줄 수도 있고, 일까지만 줄 수도 있고, 시까지만 인자로 줄 수 있게 처리
 - ✓ - c (일정 속성 수정 기간) fmd5 -c [FROM] [TO] [TARGET_DIRECTORY]
 - 탐색하고자 하는 파일의 속성 변경 시간 범위(from to)를 주는 옵션
 - 년도만 따로 줄 수도 있고, 월까지만 줄 수도 있고, 일까지만 줄 수도 있고, 시까지만 인자로 줄 수 있게 처리
 - ✓ - h (inode 값이 같은 파일, 즉 하드 링크 파일은 제외)
 - 하드 링크 파일의 경우 제외할 수 있는 옵션
 - ✓ 추가 구현 옵션 : 위 -a, m, c, h 외 추가로 쓸 수 있는 옵션으로 -s, -sa, -sm, -sc, -sh, -sah, -smh, -sch 와 함께 아래 옵션은 한 개씩만 사용. 프롬프트에서 옵션 입력 방식은 자율적으로 구현하되 모든 옵션에 대한 에러처리는 반드시 해야 함.
 - - d (디렉토리 깊이 제한) : -d [DEPTH]
 - BFS 탐색 시 depth 제한
 - - n (파일 개수 제한) : -n [COUNT]
 - 탐색 결과는 파일의 크기가 큰 것부터 n개만 출력
 - -e (특정 디렉토리 제외) : -e [DIR]
 - BFS 탐색 시 특정 디렉토리는 제외. [DIR]는 최대 3개까지는 지원되어야 함

- -p (특정 권한이 있는 파일만 탐색) : -p [mode] . 예 -p 644

- 라) 내장명령어 3, exit

- ✓ 프로그램을 종료시키는 명령어
- ✓ exit 명령어 실행 시 명령어를 실행시키는 프로세스, ssu_clean 프로세스 모두가 종료되어야 함

(예시 16). exit 명령어 출력 예시

```
su@su-virtual-machine:~/2022-1_LSP/P2$ ./ssu_clean
20220000> exit
Prompt End
su@su-virtual-machine:~/2022-1_LSP/P2$
```

- 마) 내장명령어 4, help

- ✓ 명령어 사용법을 출력하는 명령어(학생들이 구현한 내장 명령어는 모두 포함시켜야 함)

(예시 17). help 명령어 출력 예시

```
su@su-virtual-machine:~/2022-1_LSP/P2$ ./ssu_clean
20220000> help
Usage:
> fmd5/fsha -s [FILE_EXTENSION] [MINSIZE] [MAXSIZE] [TARGET_DIRECTORY]
>> [SET_INDEX] [OPTION ... ]
    [OPTION ... ]
    d [LIST_IDX] : delete [LIST_IDX] file
    i : ask for confirmation before delete
    f : force delete except the recently modified file
    t : force move to Trash except the recently modified file
    .....
> help
> exit
20220000>
```

○ make와 Makefile

- make : 프로젝트 관리 유틸리티

- ✓ 파일에 대한 반복 명령어를 자동화하고 수정된 소스 파일만 체크하여 재컴파일 후 종속된 부분만 재링크함
- ✓ Makefile(규칙을 기술한 파일)에 기술된 대로 컴파일 명령 또는 쉘 명령을 순차적으로 실행함

- Makefile의 구성

- ✓ Macro(매크로) : 자주 사용되는 문자열 또는 변수 정의 (컴파일러, 링크 옵션, 플래그 등)
- ✓ Target(타겟) : 생성할 파일
- ✓ Dependency(종속 항목) : 타겟을 만들기 위해 필요한 파일의 목록
- ✓ Command(명령) : 타겟을 만들기 위해 필요한 명령(shell)

Macro

Target : Dependency1 Dependency2 ...

<-Tab->Command 1

<-Tab->Command 2

<-Tab->...

- Makefile의 동작 순서

- ✓ make 사용 시 타겟을 지정하지 않으면 제일 처음의 타겟을 수행
- ✓ 타겟과 종속 항목들은 관습적으로 파일명을 명시
- ✓ 명령 항목들이 충족되었을 때 타겟을 생성하기 위해 명령 (command 라인의 맨 위부터 순차적으로 수행)
- ✓ 종속 항목의 마지막 수정 시간(st_mtime)을 비교 후 수행

- Makefile 작성 시 참고사항

- ✓ 명령의 시작은 반드시 Tab으로 시작해야함
- ✓ 한 줄 주석은 #, 여러 줄 주석은 ww
- ✓ 자동 매크로 : 현재 타겟의 이름이나 종속 파일을 표현하는 매크로

매크로	설명
-----	----

\$?	현재 타겟보다 수정 시간이 최신인 종속 항목 목록
^	현재 타겟의 모든 종속 항목 목록(중복 제거)
<	첫 번째 종속 항목
*	확장자 규칙에서 확장자를 제외한 파일 이름 (stem). 패턴규칙에서 확장자를 제외한 파일이름 (예: test.c → test)
@	현재 타겟의 이름

- Makefile 작성 예시

- ✓ 단일 소스 파일을 컴파일하는 규칙에서는 첫 번째 종속 항목을 의미하는 \$<를 사용하는 것이 일반적이며, 링크 단계처럼 여러 종속 파일을 함께 사용하는 경우에는 \$를 사용하는 것이 관례임.

(예시 18). Makefile	(예시 19). 매크로를 사용한 경우	(예시 20). 자동 매크로를 사용한 경우
<pre>test : test.o add.o sub.o gcc test.o add.o sub.o -o test test.o: test.c gcc -c test.c add.o: add.c gcc -c add.c sub.o: sub.c gcc -c sub.c clean : rm test.o rm add.o rm sub.o rm test</pre>	<pre>OBJECTS = test.o add.o sub.o TARGET = test CC = gcc \$(TARGET) : \$(OBJECTS) \$(CC) -o \$(TARGET) \$(OBJECTS) test.o: test.c \$(CC) -c test.c add.o: add.c \$(CC) -c add.c sub.o: sub.c \$(CC) -c sub.c</pre>	<pre>OBJECTS = test.o add.o sub.o TARGET = test CC = gcc \$(TARGET) : \$(OBJECTS) \$(CC) -o \$@ \$^ test.o: test.c \$(CC) -c \$< add.o: add.c \$(CC) -c \$< sub.o: sub.c \$(CC) -c \$<</pre>

- (예시 21). Makefile 수행 예시

<pre>oslab@a-VirtualBox:~\$ make gcc -c test.c gcc -c add.c gcc -c sub.c gcc test.o add.o sub.o -o test oslab@a-VirtualBox:~\$</pre>	<pre>oslab@a-VirtualBox:~\$ make clean rm test.o rm add.o rm sub.o rm test oslab@a-VirtualBox:~\$</pre>
--	---

○ 과제 구현에 필요한 함수 (필수로 사용해야 하는 것은 아님)

- 1. memset : s가 가리키는 메모리 영역의 처음 n바이트를 상수 바이트 c로 채우는 함수

```
#include <string.h>
void *memset(void *s, int c, size_t n);

a pointer to the memory area s.
```

- 2. strtok : 특정 문자 기준으로 문자열을 분리하는 함수

```
#include <string.h>
char *strtok(char *restrict str, const char *restrict delim);

return a pointer to the next token, or NULL if there are no more tokens.
```

- 3. strstr : 문자열에서 부분 문자열의 위치를 찾는 함수

```
#include <string.h>
```

```
char *strstr(const char *haystack, const char *needle);
```

return a pointer to the beginning of the located substring, or NULL if the substring is not found. If *needle* is the empty string, the return value is always *haystack* itself.

○ 프로그램 구현 시 유의 사항

- ssu_clean의 모든 출력 내용에 대해서 출력 형식 미 준수 시 감점(공백 또는 탭은 제외) 50% 감점
- 본 명세에서 특별하게 설명하지 않는 내용은 자율적으로 스스로 알아서 구현
- 일부 예시는 명세와 다소 다를 수 있음. 명세 기준으로 구현.

○ 구현 배점

- 프롬프트 출력 [1점]
- 내장명령어 `fmd5 -s [FILE_EXTENSION] [MINSIZE] [MAXSIZE] [TARGET_DIRECTORY](-a, -m, -c, -h 옵션 포함)과 [SET_IDX] [OPTION ... ]`의 모든 기능 및 옵션 [15점]
- 내장명령어 3, `exit` [1점]
- 내장명령어 4, `help` [1점]
- 추가 기능 옵션 `-d, -n, -e, -p` [위 기능이 모두 구현된 상태에서 옵션당 20점]
- `gettimeofday()` 등으로 탐색시간 측정 [2점]
- 다음 옵션을 추가할 경우 옵션당 10점, 최대 40점까지 부여 가능. 새로운 옵션을 추가할 경우 내장명령어 `help`에도 설명이 들어가야 하며, 추가한 옵션 기능을 상세하게 설명하고, 예시를 넣어 테스트한 내용을 보고서에 포함시켜야 함. 제시된 기능을 구현하되 명세에 없는 구현 방법이나 내용은 스스로 판단.
- ✓ 1. `-u` : 세트 내 우선 보존 규칙 지정. 예: `-u newest|oldest|pathshort|pathlong`
 - 중복 파일 세트를 찾은 뒤, `f`나 `t` 같은 삭제 동작에서 어떤 파일을 남길지 기준을 바꾸는 옵션으로 지금 명세는 `f`, `t`에서 “가장 최근 수정 파일을 남김”으로 고정되어 있음. 이것을 일반화
 - `f` 또는 `t` 옵션과 함께 사용되며, 지정한 기준에 따라 하나의 파일만 남기고 나머지는 삭제 또는 이동
 - 잘못된 OPTION 입력 시 에러 처리 후 `>>` " 출력하고 사용자 입력 대기
 - `- newest` : 가장 최근 수정된 파일을 남김 (기본값, 기존 `f/t` 옵션과 동일)
 - `- oldest` : 가장 오래된 파일을 남김
 - `- pathshort` : 절대 경로 길이가 가장 짧은 파일을 남김
 - `- pathlong` : 절대 경로 길이가 가장 긴 파일을 남김
- ✓ 2. `-r` : 결과 정렬 또는 정렬 기준 선택. 예: `-r size_desc, -r mtime_desc, -r path_asc`
 - 현재 명세는 세트 정렬 기준이 파일 크기, 최단 절대경로 길이, 해시값 순으로 세트를 정렬하게 되어 있음. 여기에 사용자가 정렬 기준을 선택 (힌트 : 링크드 리스트로 저장한 결과를 정렬 가능한 구조로 변경)
- ✓ 3. `-o` : 결과를 파일로 저장. 예: `-o [FILE]`
 - 중복 파일 탐색 결과를 지정한 파일(FILE)에 저장
 - 출력 형식은 표준 출력과 동일해야 함
 - 파일이 존재하지 않을 경우 새로 생성하며, 존재할 경우 기존 내용을 덮어씀
 - 파일 생성 또는 쓰기 권한이 없는 경우 에러 처리 후 프롬프트 출력
- ✓ 4. `-x` : 최소 중복 개수 제한. 예: `-x [COUNT]`
 - 중복 파일 세트 중에서 파일의 개수가 COUNT 이상인 세트만 출력
 - COUNT는 2 이상의 정수만 허용
 - 조건을 만족하지 않는 세트는 출력하지 않음
 - 잘못된 COUNT 입력 시 에러 처리 후 프롬프트 출력
 - (힌트 : 해시 그룹화 후 세트 크기 계산, 필터링, 출력 번호 재부여 필요)
- ✓ `-u` 옵션은 `f` 또는 `t` 옵션과 함께 사용할 수 있음
- ✓ `-x, -o` 옵션은 `-s, -a, -m, -c, -h` 옵션과 함께 사용 가능
- ✓ 추가 옵션은 중복 사용 불가

○ 필수 구현 사항

- 프롬프트 출력

- fmd5(또는 sha) -s 옵션 외 -a, -m, -c, -h 옵션
- [SET_IDX] [OPTION ...]의 모든 옵션
- help, exit 내장명령어
- gettimeofday() 등으로 전체 프로그램 수행 시간이 아니라, 주어진 옵션대로 진행한 탐색시간을 측정하여 출력(예시에는 없음)
- makefile (위 예. 두 번째 형태로 매크로 사용) 작성 (makefile 포함되지 않을 경우 -50점 감점)
- 탐색은 반드시 BFS로 구현해야 하며, 탐색된 결과는 반드시 링크드리스트로 저장 (절대 동적 배열 등 사용 불가). BFS 탐색과 링크드리스트 형태로 탐색된 결과를 저장하지 않을 경우 본 과제는 0점 처리.

○ 제출 기한 : 5월 17일(일) 오후 11시 59분 59초